PRACTICAL ASPECTS OF DEEP LEARNING

OUTLINE

- Setting up your machine learning application
  - Train, development, test sets
  - Bias / variance
- Regularizing your network
  - Regularization and overfitting
  - Dropout regularization
  - Other regularization methods
- Setting up your optimization problem
  - Normalizing inputs
  - Vanishing, exploding gradients
  - weight initialization for deep networks
  - Numerical approximation of gradients
  - Gradient checking

Learning Objectives

- Give examples of how different types of initializations can lead to different results
- Examine the importance of initialization in complex neural networks
- Explain the difference between train/dev/test sets
- Diagnose the bias and variance issues in your model
- Assess the right time and place for using regularization methods such as dropout or L2 regularization
- Explain Vanishing and Exploding gradients and how to deal with them
- Use gradient checking to verify the accuracy of your backpropagation implementation



deeplearning.ai

Setting up your ML application

Train/dev/test sets

Applied ML is a highly iterative process

Important decisions:

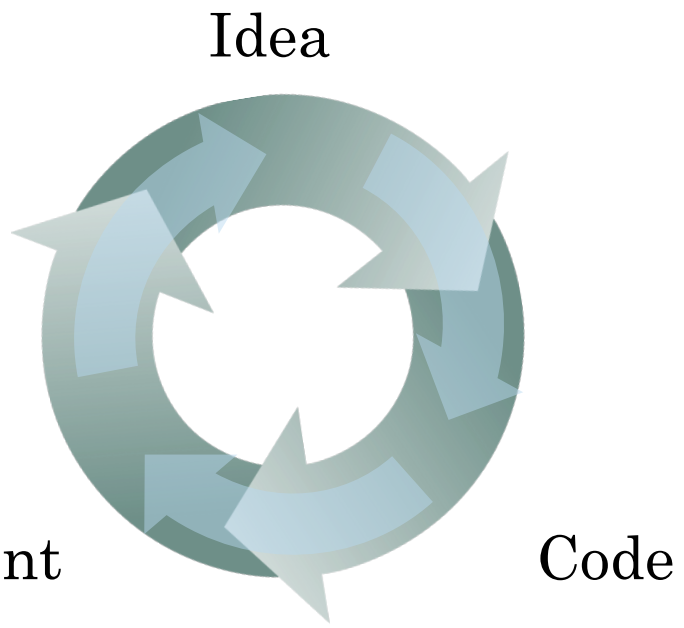
layers

hidden units

learning rates

activation functions

...



- It is impossible to choose all these parameters perfectly at first.
- ML is an iterative process: try and change parameters

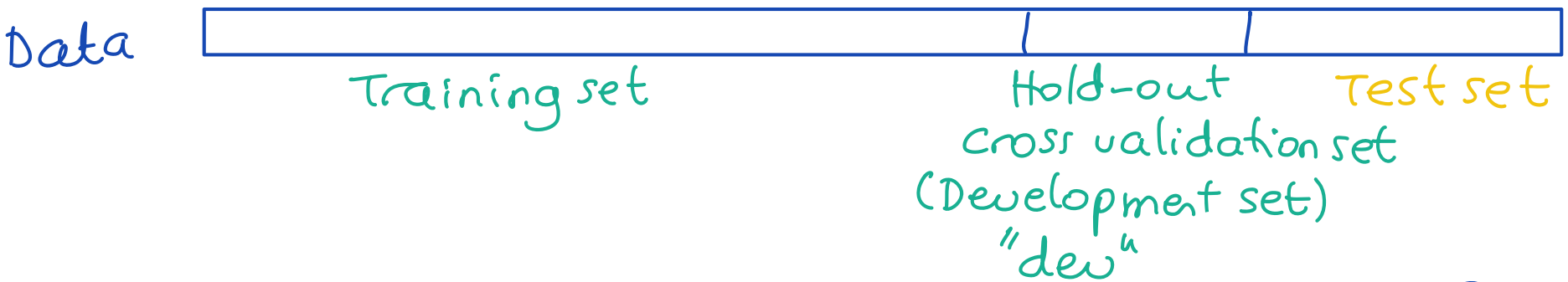
Andrew Ng

- There are many application areas of deep learning
  NLP, Vision, Speech, Structured Data (Ads, security, logistics...)
- when a researcher working in one application area starts working on another application (e.g. NLP to Vision) intuitions from one domain often do not transfer to other domains.

- Factors to consider: #training data, computer configuration (CPU vs. GPU etc.)

- Using training, development and test sets will make the iteration process very efficient.

Train/dev/test sets



- Train the model using training set and see its performance on the validation set. If necessary, change some parameters and repeat.
- Finally, use the test set to evaluate your "best model" from the previous step. This will give an unbiased estimate of how well your algorithm is doing.

Andrew Ng

- Previous era: used data splits like:

  70% / 30%
  train test

  60% / 20% / 20%
  train dev test

- These ratios were perfectly reasonable rules of thumb if your data consisted of 100, 1000 or up to 10000 samples.

- Big data era: Data size $\geq$ 1 Million samples
  - Percentage of dev and test sets have become much smaller percentage of the total data
  - Dev set is needed to quickly make design choices. 20% of data may not be needed.
  - e.g. for data size of 1M samples, 10000 samples (1%) may be sufficient for dev set, and test set (1%)

    1.000.000 10.000 10.000
    dev test

    $\Rightarrow$ 98% / 1% / 1% may be enough
    99.5% / 0.25% / 0.25%

- In deep learning era, more people are training on mismatched train/test distributions.

Goal: You're building an app that lets users upload a lot of pictures and your goal is to find pictures of cats in order to show your users.

Mismatched train/test distribution

Training set:
Cat pictures from webpages

high-resolution, sharp

Dev/test sets:
Cat pictures from users using your app

low-resolution, blurry

different distributions

- Make sure that dev and test sets come from the same distribution, for reliable evaluation of performance.
- Not having a test set might be okay. (Only dev set.)
  - Test set is needed to get an unbiased estimate of the performance of the final network.
  - If you don't need that unbiased estimate, it might be okay to not have a test set:

Andrew Ng

train / dev set

↳ maybe called as "test" set

then, people may be overfitting to the dev set, so be cautious.

- Splitting the data as train / dev / test will help to determine the bias and variance of the algorithm so the algorithm may be improved more efficiently.

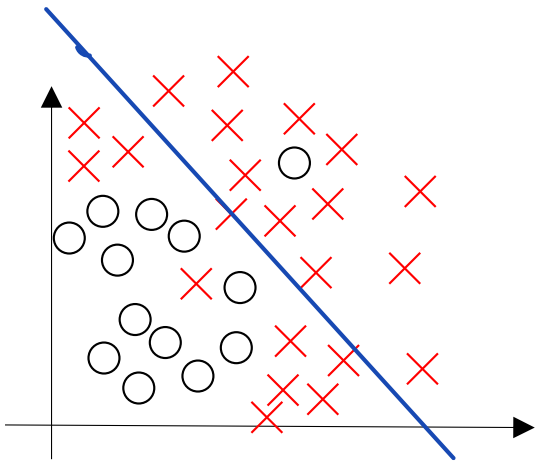


deeplearning.ai

Setting up your ML application

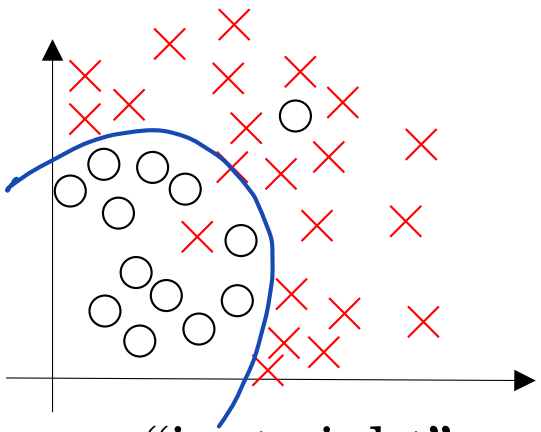
Bias/Variance

Bias and Variance

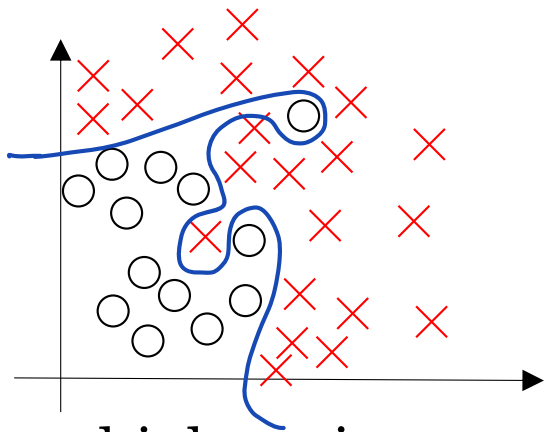


high bias

underfitting the data



"just right"



high variance

e.g. a deep neural net with many hidden units

overfitting the data

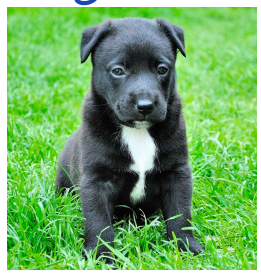
Andrew Ng

- How can we understand bias and variance for high dimensional data?

Bias and Variance

Cat classification





	case 1	case 2	case 3	case 4
Train set error:	1%	15%	15%	0.5%
Dev set error:	11%	16%	30%	1%
	high variance (overfit)	high bias (underfit)	high bias high variance	low bias low variance

Humans can recognize with ≈0% error.

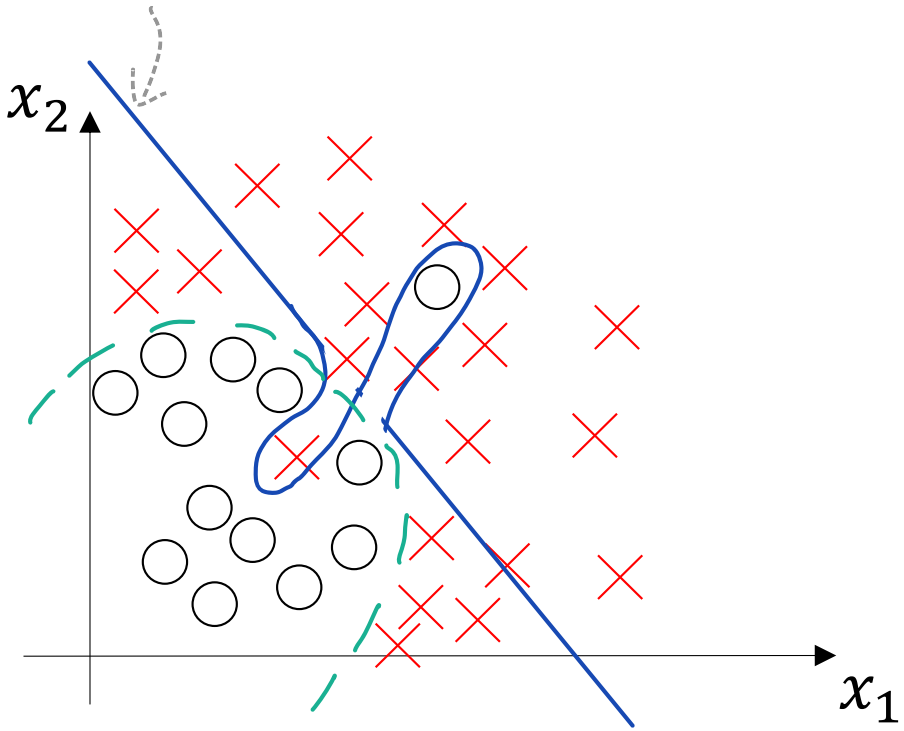
Optimal error (Bayes error) ≈ 0%

Andrew Ng

* If optimal error were 15%, case 2 wouldn't be an underfit. For example if images were blurry, humans could only achieve 15%.
* Summary:
    * Looking at training error will tell us how well we are fitting to training data, i.e. if there is a high bias problem.
    * Looking at the difference between dev set and train set error, we can understand if there is a high variance problem.

Assumptions: • Bayes error is quite small
• Training and dev sets come from the same distribution.

High bias and high variance



- High bias: not fitting training data well since it is mostly linear. (not as good as the green quadratic curve)
- High variance: since it overfits the two exampler in the middle.

Andrew Ng



deeplearning.ai

Setting up your ML application

Basic "recipe" for machine learning

- How can we improve our algorithm if it has high bias or high variance issues?

Basic "recipe" for machine learning

(1) When training a neural network, first check if there is high bias by looking at training data performance.

(2) If there's high bias, choose a bigger network (more layers, more units per layer) or train longer (advanced optimization algorithms) or choose another neural network architecture better suited to this problem.

(3) Try (1) and (2) until you fit the training set well.

(4) Determine if there is high variance problem by looking at the dev set performance. Does your model generalize well to a good dev set performance?

Andrew Ng

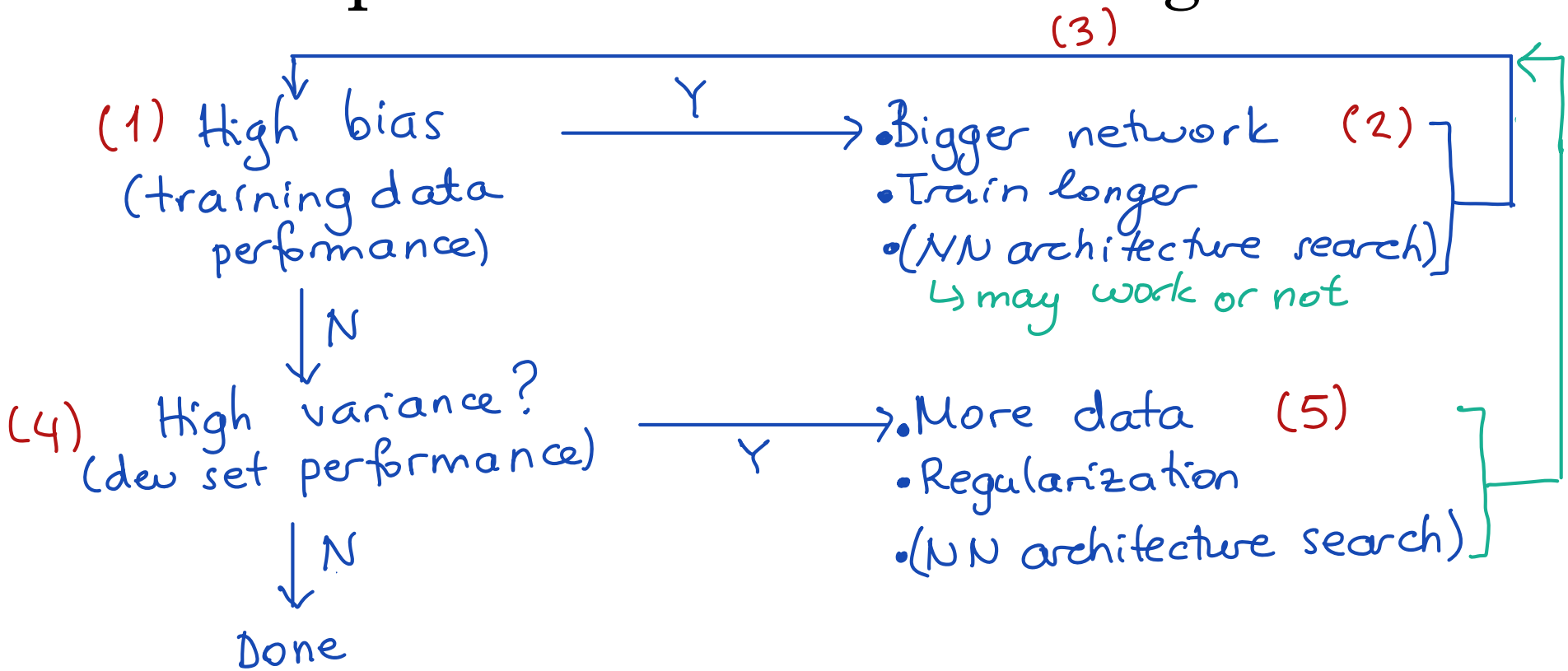
Basic "recipe" for machine learning

(5) If there's high variance, you can try to get more data, or use regularization or try different architectures.

- If you have a high bias problem, getting more data will not help.
- Pre-deep learning era: there was a bias-variance trade-off: increasing bias reduced variance etc. There were not enough tools to just reduce bias or just reduce variance without hurting the other one.
- Modern deep learning (big data) era: you can train a bigger network with more data to reduce bias, and this doesn't hurt variance as long as you regularize properly. Getting more data reduces variance but does not hurt your bias much.

Andrew Ng

Basic recipe for machine learning



Andrew Ng



deeplearning.ai

Regularizing your neural network

Regularization

- If your neural network is overfitting your data, then you have a high variance problem.
  - Then one of the first things to try is regularization.
  - Another solution is to get more data but you can't always get more data.

- First, let us look at the case of logistic regression.

Logistic regression

$\min_{w,b} J(w,b)$, $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

$$J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

regularization parameter

add a regularization term

L_2 regularization: $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$ (square of the Euclidean norm of w)

L_1 regularization: $\frac{\lambda}{2m} \sum_{i=1}^{n_x} |w| = \frac{\lambda}{2m} \|w\|_1$

$\Rightarrow w$ will be sparse (will have a lot of zeros in it)
$\Rightarrow$ will help to compress the model parameters

Andrew Ng

* Why don't we regularize the bias term by adding another regularization term?
  - Most of the parameters are in the $w \in \mathbb{R}^{n_x}$, whereas $b \in \mathbb{R}$ is just a number. In practice regularizing b does not make much difference.

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m}\|w\|_2^2 + \underbrace{\frac{\lambda}{2m} b^2}_{\text{omit}}$$

- In practice, L_2 regularization is used more often than L_1 regularization.
- Regularization parameter λ is determined by cross-validation in practice. It is a hyperparameter to be tuned.

- Next: how do we regularize a neural network?

Neural network

regularization term

$$J(W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^{L} \| W^{[l]} \|_F^2$$

where $\| W^{[l]} \|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (W_{i,j}^{[l]})^2$, $W^{[l]} : (n^{[l]}, n^{[l-1]})$

↳ Frobenius norm $\| \cdot \|_F^2$ of a matrix

(Note: The rows "i" of the matrix is the number of neurons in the current layer $n^{[l]}$ and the columns "j" of the weight matrix is equal to the number of neurons in the previous layer $n^{[l-1]}$.)

Andrew Ng

- How do we implement gradient descent with regularization?

  update $dW^{[l]}$ as follows:

$$\frac{\partial J}{\partial W^{[l]}} = dW^{[l]} = (\text{from backprop}) + \underbrace{\frac{\lambda}{m} W^{[l]}}_{\text{regularization term}}$$

$$W^{[l]} := W^{[l]} - \alpha\, dW^{[l]}$$

$$W^{[l]} := W^{[l]} - \alpha \left[(\text{from backprop}) + \frac{\lambda}{m} W^{[l]} \right]$$

$$= W^{[l]} - \frac{\alpha\lambda}{m} W^{[l]} - \alpha\, (\text{from backprop})$$

$$(\text{"weight decay"}) = \underbrace{\left(1 - \frac{\alpha\lambda}{m}\right)}_{<1} W^{[l]} - \alpha\, (\text{from backprop})$$

$\Rightarrow$ weights are pushed towards zero.

- L_2 regularization is also called <u>weight decay</u>.

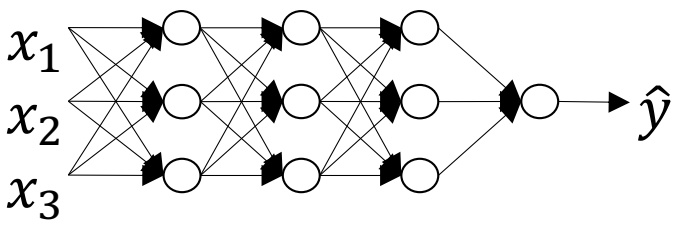


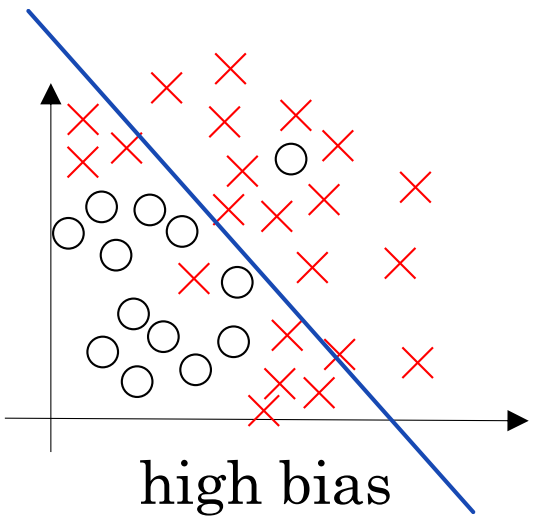
deeplearning.ai

Regularizing your neural network

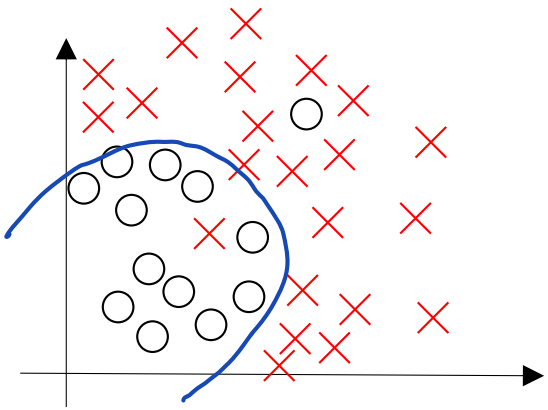
Why regularization reduces overfitting

How does regularization prevent overfitting?

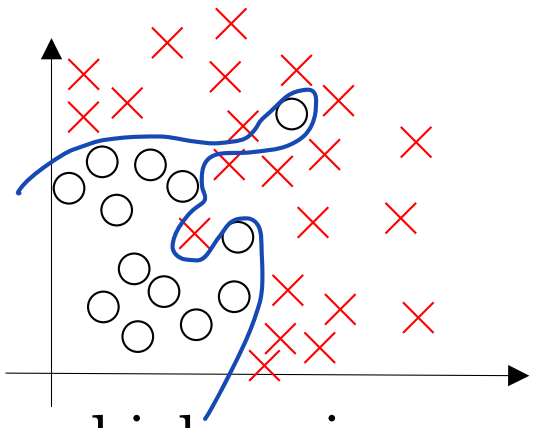




high bias



"just right"



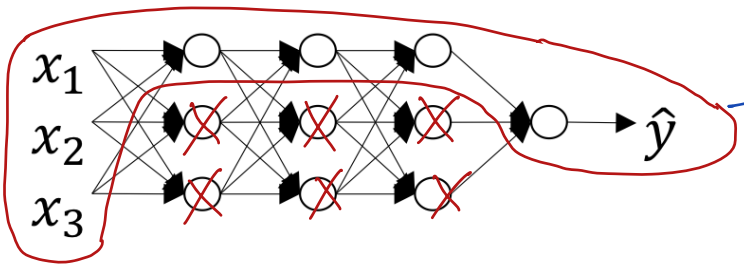
high variance

Andrew Ng

- Let us assume the neural network above is overfitting.

$$J(W^{[l]}, b^{[l]}) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \underbrace{\frac{\lambda}{m} \sum_{l=1}^{L} \|W^{[l]}\|_F^2}_{\text{regularization term}}$$

- If λ is large $\Rightarrow W^{[l]} \approx 0$



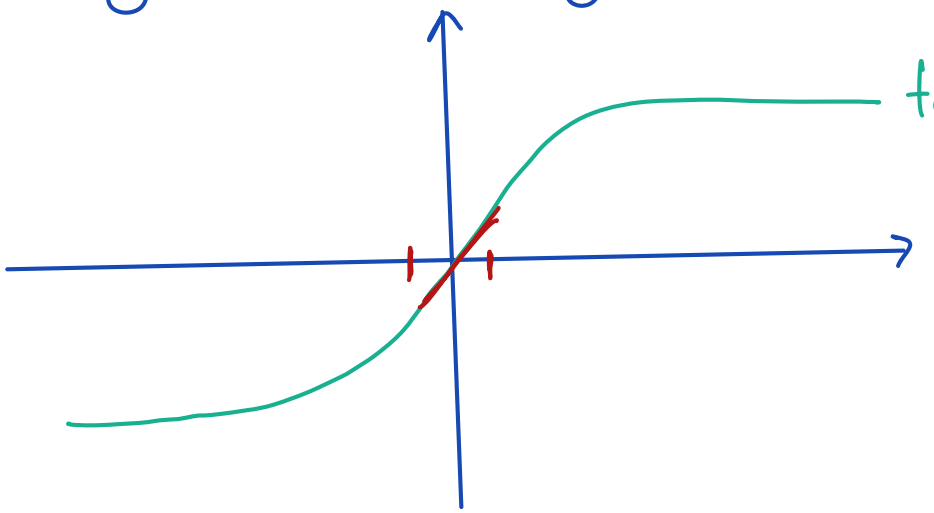
→ simplified network

$\Rightarrow$ The network will be simpler, since some hidden units will have smaller weights and have less effect.

Another intuition

How does regularization prevent overfitting?

- Let's say we are using the tanh activation function.



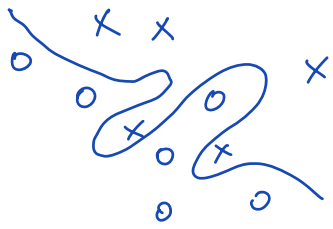
- If z is small, we are using the linear regime of the tanh function. If z is large, activation function is less linear.
- If λ is large $\Rightarrow W^{[l]}$ will be small and z will be small

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$\Rightarrow g(z)$ will be linear

$\Rightarrow$ Every layer will be approximately linear.

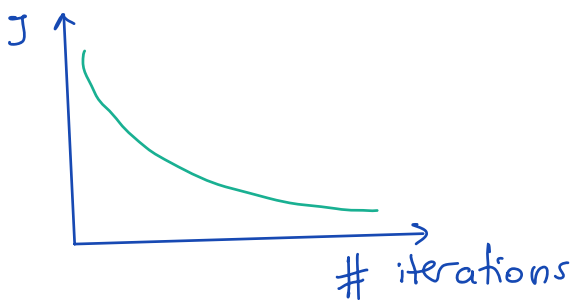
- If all layers $\approx$ linear, then whole NN will compute a linear function. So it can't find a highly non-linear decision boundary, which causes overfitting.



- Implementation Detail:

  Our cost function is of this form:

$$J(\dots) = \sum_i \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{m} \sum_l \|w^{[l]}\|_F^2$$



- During gradient descent we plot the cost function J and we want to see that J decreases after every iteration.

- With regularization you should plot the new definition of J (with regularization term). Otherwise J may not decrease after every iteration.



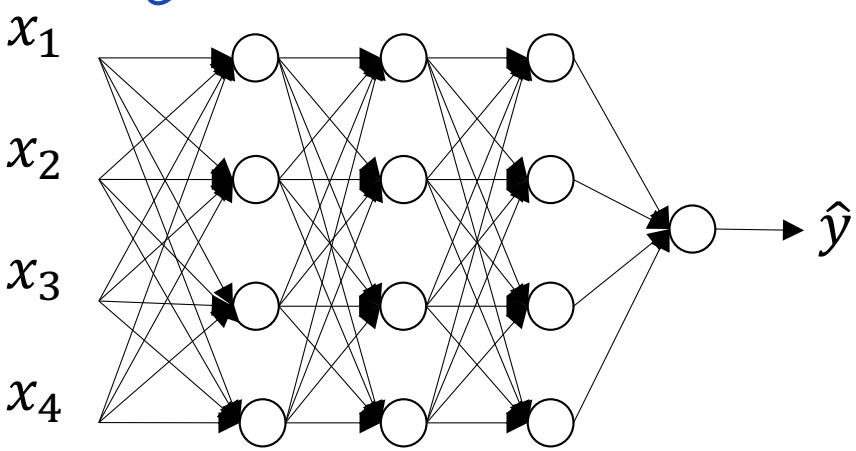
deeplearning.ai

Regularizing your neural network

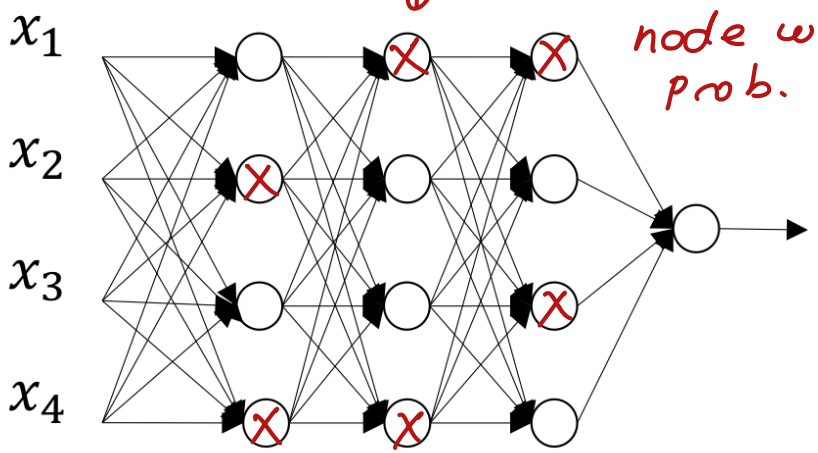
Dropout regularization

Dropout regularization

- Let's assume the below neural network is overfitting during training.



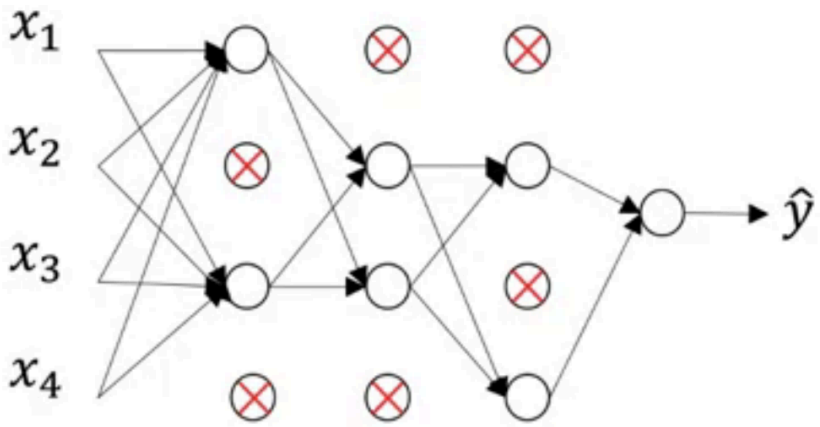
- How do we apply dropout regularization? e.g. 0.5 Toss a coin and eliminate a node with prob. 0.5.



- We will set a probability of eliminating a node in the network. by going through each layer of the network

Andrew Ng

- Eliminate all ingoing and outgoing weights from that node as well and this will give a much smaller network, and perform backpropagation on that training example.
- A diminished network:



- On different examples, toss a different set of coins again and keep a different set of nodes. For each training example train the network using one of these smaller networks.
- At each iteration of gradient descent, different nodes are zeroed-out.
- Using a <u>smaller network</u> has a <u>regularizing effect</u>.

- There are several ways of implementing dropout. One of them is the "inverted dropout".

Implementing dropout ("Inverted dropout")

Illustrate with layer $l=3$. : keep-prob = 0.8

```
d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep-prob
```

↳ dropout vector : boolean array of 1's and 0's.

```
# keep-prob : probability that a hidden unit will be kept during
  forward prop. and backward propagation.
a3 = np.multiply(a3, d3)   # elementwise multiplication
# a3 *= d3 → equivalent, zero-out some activations.
a3 /= keep-prob   #scale : 50 units in total → 10 units shut-off
```

$$z^{[4]} = W^{[4]} a^{[3]} + b^{[4]}$$

↳ reduced by 20%.

in order not to reduce the expected value of $a^{[3]}$, divide it by 0.8 ⇒ Hence the name "inverted dropout".

Andrew Ng

Making predictions at test time

* Using inverted dropout makes test time easier since there's less scaling problem. It is the most common dropout technique.

- Given $a^{[0]} = x$ on which to make a prediction (test example)
- Don't use dropout at test time, since we don't want our predictions to be random (or contain noise).

$$z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$$
$$a^{[1]} = g^{[1]}(z^{[1]})$$
$$z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$$
$$a^{[2]} = g^{[2]}(z^{[2]})$$
$$\vdots$$
$$\hat{y}$$

- Don't use the scaling step at test time (/keep-prob)

Andrew Ng

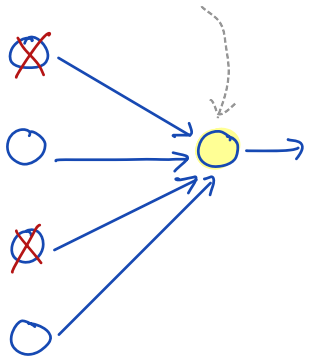


deeplearning.ai

Regularizing your neural network

Understanding dropout

- Why does drop-out work as a regularizer?
  Let's think in terms of a single unit:

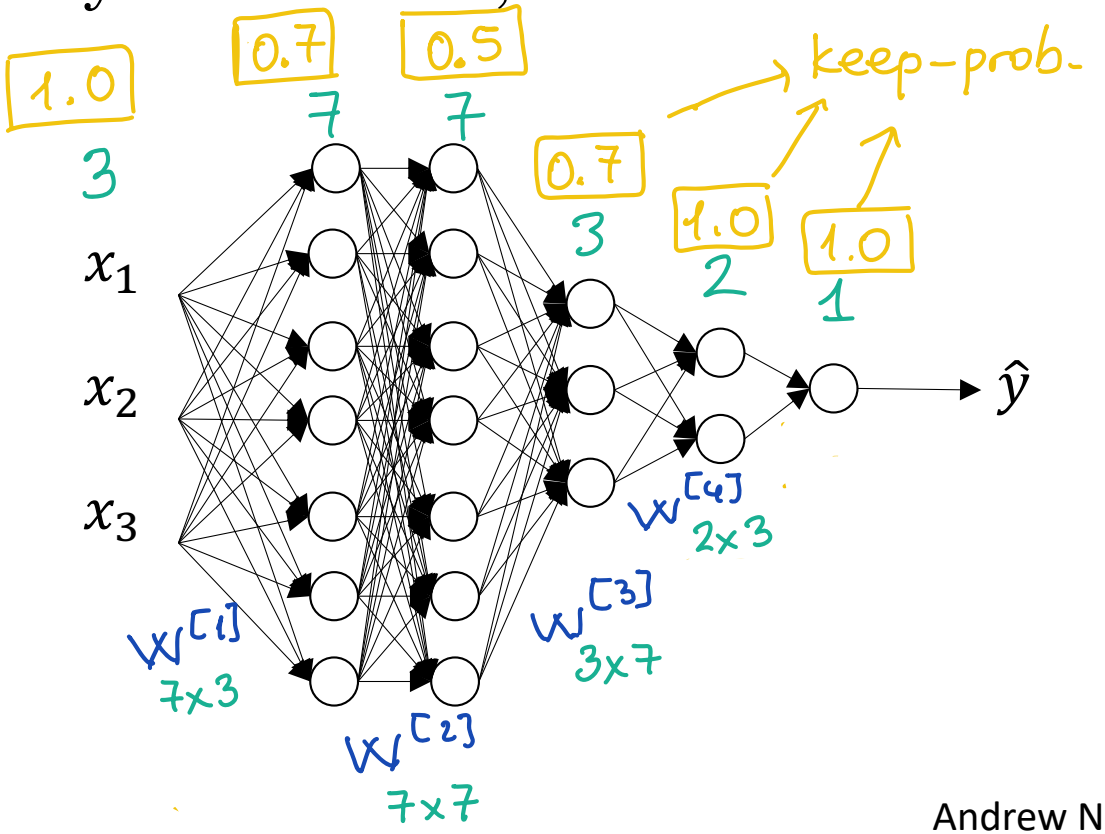


  - This unit takes 4 inputs and generates an output. But with dropout some input units can be randomly eliminated at each iteration.
  - Hence, the yellow unit can not rely on any one feature because any of its inputs can go away any minute.
  - Therefore, the yellow unit will not put too much weight on any one input, but spread out its weights over all 4 input units. It will put a little weight on all input units.
  - This will result in a shrinking of the weights. Has similar effects as L_2 regularization.
  - Dropout can formally be shown to be an adaptive form of L_2 regularization.

Why does drop-out work? (as a regularizer)

Intuition: Can't rely on any one feature, so have to spread out weights.

- It is possible to change keep-prob. parameter at each layer.



Andrew Ng

- $W^{[2]}$ has size 7x7 and has the highest number of parameters. Hence, we can choose the keep_prob. parameter a little lower for this layer, to prevent overfitting.
- If you don't worry about overfitting for a layer, you can set keep_prob = 1.0 for that layer.

- One of the first applications of drop-out was for computer vision, since the input image size is big, and you have a lot of weights, data may not be sufficient.

- Dropout is a regularization technique and it prevents overfitting. Unless there is overfitting, do not use it.

Down-side of Dropout

- The cost function J is not well-defined. Plotting J versus #iterations is a good debugging tool, too see J decreases at each iteration, and this may not work well with dropout

- May turn off dropout or set keep_prob. = 1 and make sure J decreases monotonically. Then, turn on drop out



deeplearning.ai

Regularizing your neural network

Other regularization methods

Data augmentation

flip horizontally

random crop & zoom







4

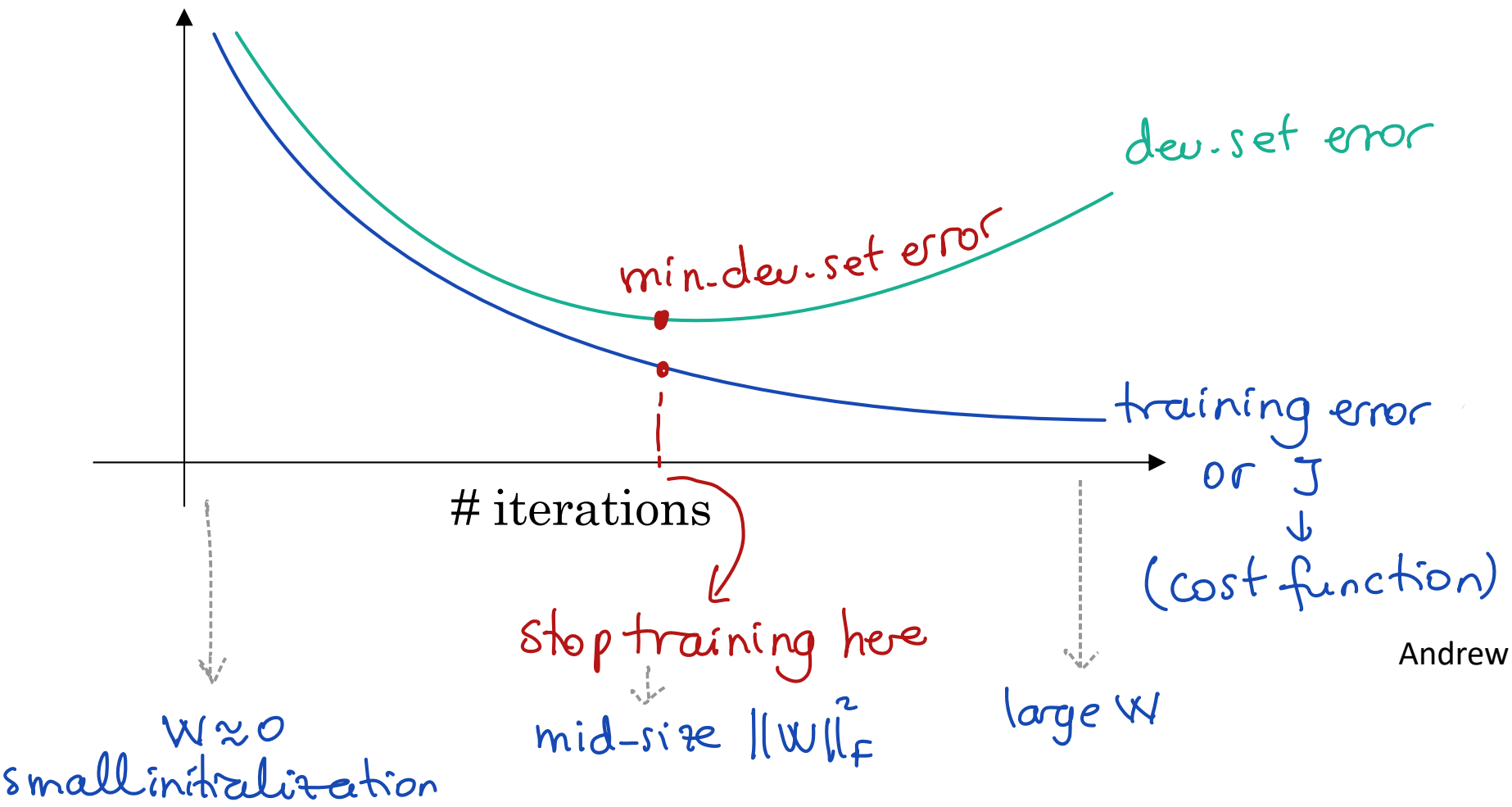


Andrew Ng

- Let's say you're training a cat classifier and you're overfitting.
- Getting more data could help, but it may not always be possible
- You can use data augmentation to create extra fake training examples:
  - horizontal flipping
  - random cropping and scaling.

- Data augmentation does not bring as much new information as collecting new data.
- We didn't flip vertically since we don't want upside-down cats.
- Data augmentation is a regularization technique

Early stopping : Run gradient descent, plot training error and development set (validation) error.



Andrew Ng

Down side of Early Stopping

In machine learning

- We want to optimize the cost function J } one task: reduce J(w,b)
  - Gradient descent etc...
- We also do not want to overfit } a seperate task
  - Regularization...

this seperation of tasks is called orthogonalization

- One down side of early stopping is that it couples these two tasks. We can't work on the two tasks independently. We're not fully optimizing J by stopping early.

- One alternative to early stopping is L_2 regularization. Then, network can be trained as long as possible. But you may need to try many regularization parameter lambda, which may be computationally expensive.



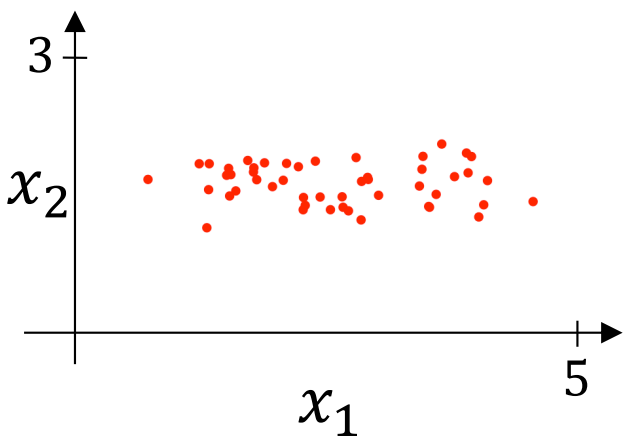
deeplearning.ai

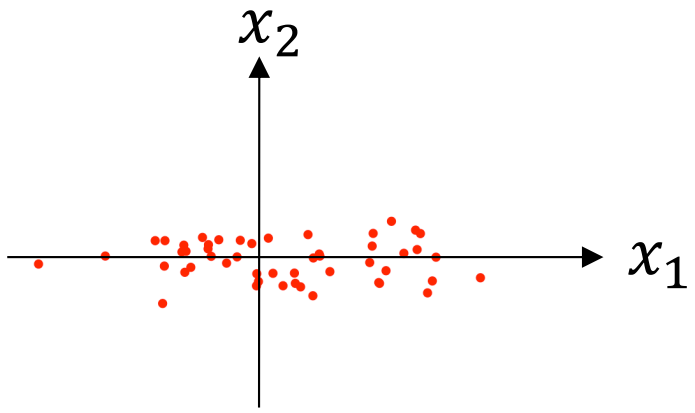
Setting up your optimization problem

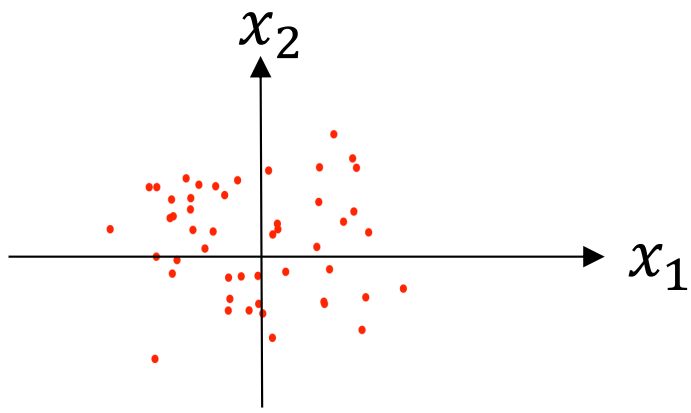
Normalizing inputs

- It is a technique to speed up the training process.

Normalizing training sets : done in two steps







20 features:

$$\underline{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Step 1: Subtract mean

$$\underline{\mu} = \frac{1}{m} \sum_{i=1}^{m} \underline{x}^{(i)}$$

$$\underline{x} := \underline{x} - \underline{\mu}$$

Step 2 : Normalize variance

$$\sigma = \frac{1}{m} \sum_{i=1}^{m} (x^{(i)})^2$$

$$\underline{x} \;/= \sigma$$

Andrew Ng

* Use the same $\underline{\mu}$ and σ to normalize the test set.

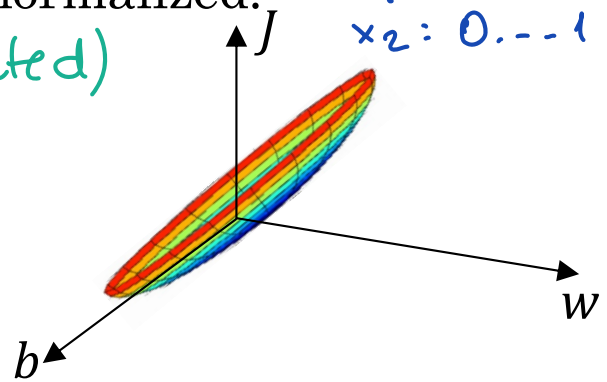
Why normalize inputs?

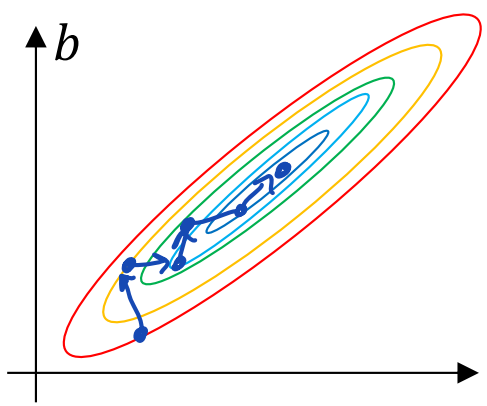
$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right)$$

Unnormalized:

range
x_1 : 1...1000
x_2 : 0...1

(Elongated)



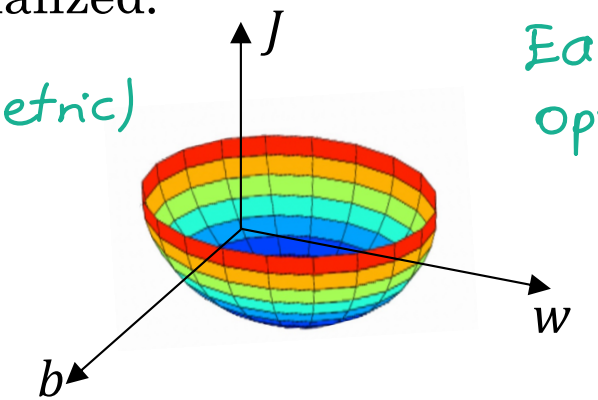


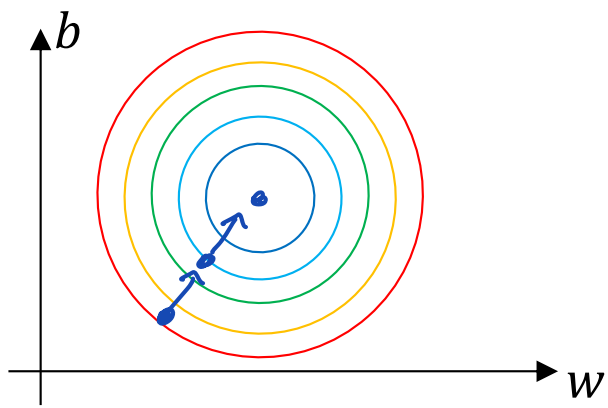
- Need a small learning rate
- Takes longer to converge (oscillates)

Normalized:

(More symmetric)

Easier to optimize.





- Gradient descent takes directly to the minimum.
- Faster to converge.

Andrew Ng

- If you have 3 features and if their ranges are as follows, it will be fine:

  x_1 : 0 ... 1

  x_2 : -1 ... 1

  x_3 : 1 - 2

- but if ranges are as follows, it is better to normalize since it hurts the optimization algorithm.

  x_1 : 1 ... 1000

  x_2 : 0 ... 1

eg

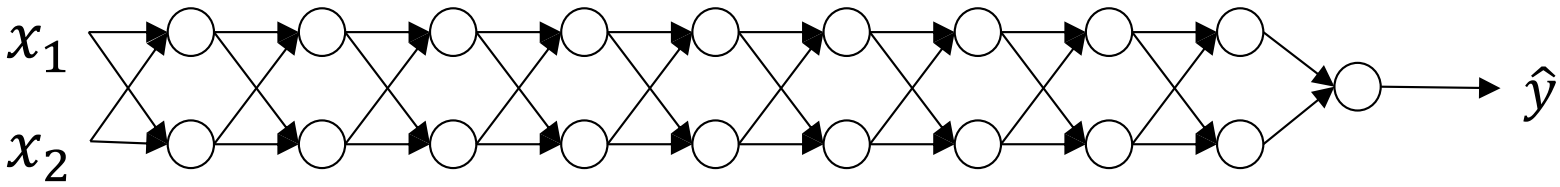


deeplearning.ai

Setting up your optimization problem

Vanishing/exploding gradients

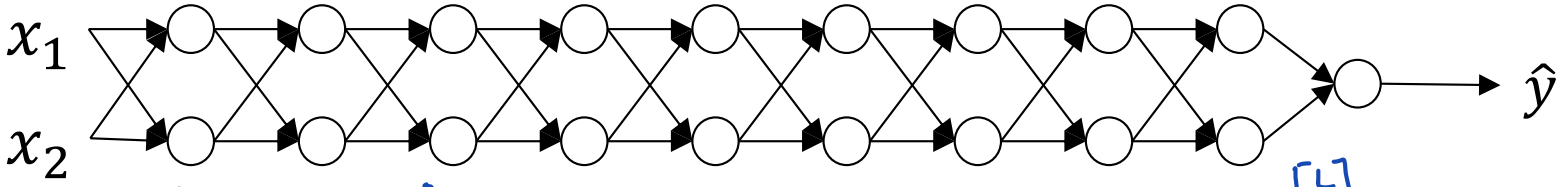
Vanishing/exploding gradients



- When training a very deep network the derivatives or slopes can sometimes get very, very big or very, very small maybe even exponentially small and this makes training difficult.
- Careful choices of the random weight initialization can significantly reduce this problem.

Andrew Ng

Vanishing/exploding gradients : A deep network



Params: $W^{[1]}$ $W^{[2]}$ $W^{[3]}$ ...

$W^{[L]}$

Activation: $g(z) = z$ (linear activation) Assume $b^{[l]} = 0$ for simplicity

Output: $\hat{y} = W^{[L]} W^{[L-1]} W^{[L-2]} \ldots W^{[3]} W^{[2]} W^{[1]} x$

$z^{[1]} = W^{[1]} x$

$a^{[1]} = g(z^{[1]}) = z^{[1]}$

$a^{[2]} = g(z^{[2]}) = g(W^{[2]} a^{[1]})$

$a^{[3]}$

Andrew Ng

Let's say $W^{[l]} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$ a little larger than 1 times identity.

Then,

$$\hat{y} = W^{[L]} \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}^{L-1} x \approx (1.5)^{L-1} x$$

→ grows exponentially

has different dimension

if L is large for a very large network, then $\hat{y}$ will be large

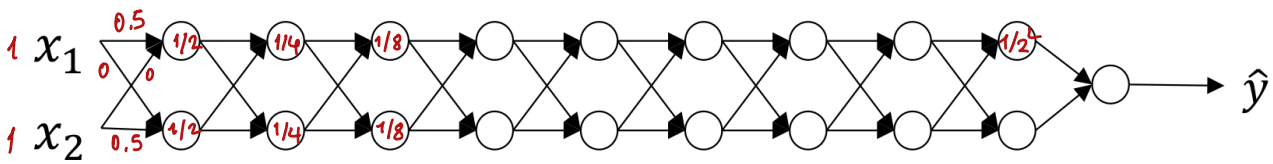
$\hat{y}$ will explode.

if $W^{[l]} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$

$$x \approx (0.5)^{L-1} x$$

decrease exponentially

⇒ activations will decrease exponentially in a deep network

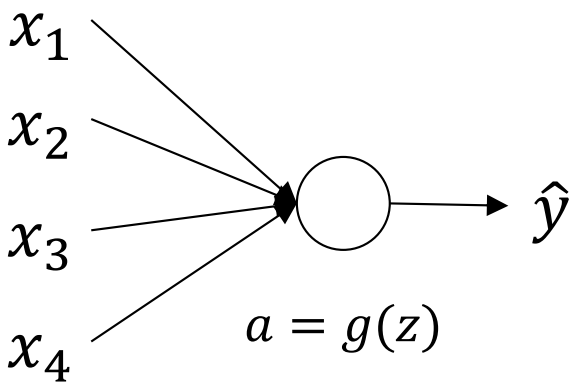


- A similar argument can be used to show that the derivatives or gradients will also increase or decrease exponentially as a function of the number of layers.
- In practice L can be about 150 layers.
- The activations can be really big or small and this makes training difficult. Gradient descent will take tiny little steps and learning will take a long time.

- A partial solution: careful choice of the random initialization for your neural network.

Weight Initialization for Deep Networks

Single neuron example



$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$ (ignore for now)

- In order not to make z too large or too small, the larger n is the smaller w_i should be, since we are summing the values.

  large $n \rightarrow$ smaller w_i

- we could set $\text{variance}(w_i) = \frac{1}{n}$, where n: # inputs to that neuron.

- In practice: $W^{[l]} = \text{np.random.randn(shape)} * \text{np.sqrt}\left(\frac{1}{n^{[l-1]}}\right)$

Andrew Ng

- If ReLU activation function is used setting $Var(w_i) = \frac{2}{n}$ may work better in practice. (He initialization)
- Note: if you multiply a random variable w_i with $sqrt\left(\frac{2}{n^{[l-1]}}\right)$ its variance becomes $\frac{2}{n^{[l-1]}}$.

- If input features or activations are roughly mean 0 and standard variance, then z will also take a similar scale.

Other variants:

- tanh activation function: multiply by $\sqrt{\frac{1}{n^{[l-1]}}}$ Xavier initialization

- Another version: multiply by $\sqrt{\frac{2}{n^{[l-1]} + n^{[l]}}}$



deeplearning.ai

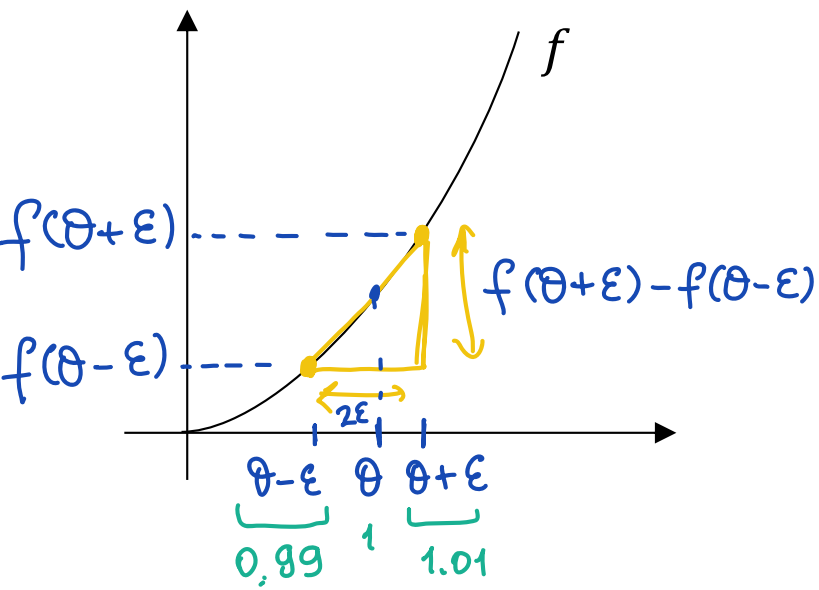
Setting up your optimization problem

Numerical approximation of gradients

Checking your derivative computation

- To make sure that your implementation of back propagation is correct, there's a test called gradient checking.
- First, let's see numerical computation of gradients.

$f(\theta) = \theta^3$



$\varepsilon = 0.01$

- Ratio of the height and width of the yellow triangle is a good estimate of the derivative at θ (centered approximation):

$$\frac{\text{height}}{\text{width}} = \frac{f(\theta+\varepsilon) - f(\theta-\varepsilon)}{2\varepsilon} \approx g(\theta) = \frac{d}{d\theta} f(\theta) = f'(\theta)$$

$$= \frac{(1.01)^3 - (0.99)^3}{2(0.01)} = 3.0001$$

$g(\theta) = 3\theta^2 = 3$ ← approximation error = 0.0001

Andrew Ng

- Formal definition of derivative:

$$f'(\theta) = \lim_{\varepsilon \to 0} \underbrace{\frac{f(\theta+\varepsilon) - f(\theta-\varepsilon)}{2\varepsilon}}$$

error for this approximation is $O(\varepsilon^2)$

if $\varepsilon = 0{,}01$

$\varepsilon^2 = 0{,}0001$

Two-sided formula is more accurate

Less accurate formula:

$$f'(\theta) = \lim_{\varepsilon \to 0} \underbrace{\frac{f(\theta+\varepsilon) - f(\theta)}{\varepsilon}}$$

Approximation error is $O(\varepsilon)$



deeplearning.ai

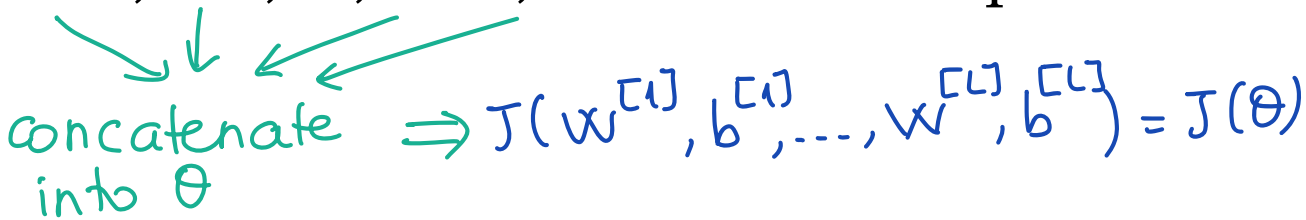
Setting up your optimization problem

Gradient Checking

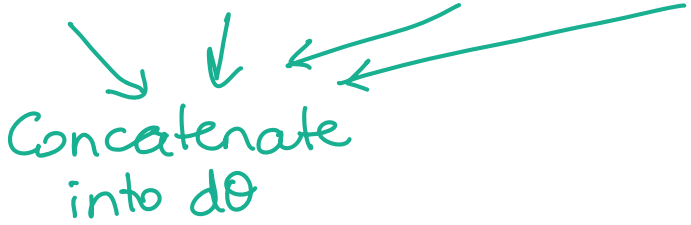
- Helps to find bugs in implementation of backpropagation.

Gradient check for a neural network

Take $W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]}$ and reshape into a big vector θ.



Take $dW^{[1]}, db^{[1]}, \ldots, dW^{[L]}, db^{[L]}$ and reshape into a big vector $d\theta$.



- Is $d\theta$ the gradient of $J(\theta)$?

Gradient checking (Grad check)

$J(\theta) = J(\theta_1, \theta_2, \dots)$, want to check $\partial J / \partial \theta_i$

for each i:

$$d\theta_{approx}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \varepsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \varepsilon, \dots)}{2\varepsilon}$$

$$\approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$

Compute this for each $\theta[i]$, then $d\theta_{approx}[i] \overset{?}{\approx} d\theta$ (check this)

to check, can use Euclidean distance:

check: $$\frac{\|d\theta_{approx} - d\theta\|_2}{\|d\theta_{approx}\| + \|d\theta\|_2}$$

for normalization

- if this is approx. 10^{-7} for $\varepsilon = 10^{-7}$, this means your computation is correct.
- 10^{-5} → check
- 10^{-3} → maybe a bug somewhere

Andrew Ng



deeplearning.ai

Setting up your optimization problem

Gradient Checking implementation notes

Gradient checking implementation notes

- Don't use in training – only to debug : Computation of gradient check ($d\theta_{approx}[i]$) takes time. Use it only when debugging. Turn it off during training.
- If algorithm fails grad check, look at components to try to identify bug.
  If $d\theta_{approx}[i]$ is different from $d\theta[i]$, then, this gives an idea about the layer the bug may belong to: $db^{[l]}$, $dW^{[l]}$
- Remember regularization. Include the regularization term in the cost function: $J(\theta) = \frac{1}{m}\sum_i \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m}\sum_l \|W^{[l]}\|_F^2$, $d\theta$ = gradient of J w.r.t θ
- Doesn't work with dropout.
  There's no clear definition of $J(\theta)$ when using dropouts, since some θ values are randomly erased.
- Run at random initialization; perhaps again after some training.
  Your gradient descent may work when $w, b \approx 0$ (at initialization) and may not work after a number of iterations. So, run gradient checking later during the iterations, as well.

Andrew Ng